PROJECT-BASED LEARNING

PHASE-II EVALUATION

FOOD FLOW: Intelligent Food Recommendation and Route Optimization System

Team ID: DSCPP-III-2026-T165

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Mentor: Dr. Pawan Kumar Mishra

1. Team and Project Information

Project: FOOD FLOW — Intelligent Food Recommendation and Route Optimization System

Team ID: DSCPP-III-2026-T165

Semester: 3rd | Domain: Data Structures & C++

Mentor: Dr. Pawan Kumar Mishra

Team Member	Student ID	Role	Module / Contribution (as listed in Phase-I source)
Ipsita Bansal	2510013430	Team Lead	Login, password and feedback module
Surya Pratap Tyagi	2510012313	Member	Delivery module
Shubhda Joshi	2510012509	Member	Dine-in module
Sehar Gupta	2510350019	Member	Takeaway and drive-through module

2. Project Overview

• The project proposes a C++ console prototype for food selection and route planning.

• Users provide food preferences, location and service type.

• Food places are to be compared using factors such as rating, distance, preparation time and user choice.

• The service options described in the source are dine-in, takeaway / drive-through and delivery.

• The proposed system also includes login, ordering, feedback and order-history features.

3. Problem Statement and Motivation

1. Users may face too many food and restaurant choices.

2. The nearest restaurant may not necessarily match a user’s preference.

3. A route-planning method is needed to determine a suitable path to a selected place.

4. Users may require different service modes, including dine-in, takeaway / drive-through and delivery.

5. Login, feedback and order history are identified as useful supporting features.

4. Objectives

1. Develop a working C++ console prototype for food selection and route planning.

2. Compare food places using rating, distance, preparation time and user preference.

3. Provide service choices for dine-in, takeaway / drive-through and delivery.

4. Plan routes using a graph representation and Dijkstra’s algorithm.

5. Organize user, restaurant, service and order-related functionality into manageable modules.

6. Include login, feedback and order-history functionality in the planned system.

5. Proposed Solution and Workflow

1. User Input: collect food choice, location and service type.

2. Food Processing: identify available food items and restaurant details.

3. Recommendation: filter and compare food options based on the stated preferences and project factors.

4. Route Planning: represent locations and routes as a graph and apply Dijkstra's algorithm to calculate a shortest path.

5. Service Selection: allow the user to select dine-in, takeaway / drive-through or delivery.

6. Final Output: show the selected food place, service option and route details.

7. Feedback: collect feedback after the service or order, as described in the proposed workflow.

Problem–Solution Mapping

Problem / Need	Proposed response
Too many choices	Filter and compare food-place options.
Nearest place may not suit	Consider rating, distance and preference.
Route planning required	Represent routes with a graph and use Dijkstra's algorithm.
Different service choices	Support dine-in, takeaway / drive-through and delivery.
User records needed	Provide login, feedback and history features.

6. Integration of Course Concepts

Course / Concept	Application in FOOD FLOW
Data Structures	Graphs represent locations and routes; Dijkstra's algorithm computes shortest paths; searching and sorting help
OOP with C++	Classes and objects represent users, restaurants and orders; functions divide the program into modules; STL co

• Graph + Dijkstra: route representation and shortest-path calculation.

• Searching / sorting: processing and comparing food-place information.

• Classes / objects / functions: modular organization of related data and operations.

• Linked lists, stacks, queues and hash maps: data-handling structures identified in the Phase-I plan.

7. System Architecture

User Input	→	**Login / Profile**	→	**Food Selection**
		↓		
Service Choice	→	**Order / Route**	→	**Feedback**

Major Components

1. Login and password.

2. Food selection and recommendation.

3. Dine-in service.

4. Takeaway / drive-through service.

5. Delivery service.

6. Route optimization.

7. Feedback and order history.

8. Technology Stack and Methodology

• Language: C++ with an object-oriented approach.

• Data structures: graph, vector, queue, stack, linked list and hash map.

• Algorithm: Dijkstra’s algorithm for shortest-route calculation.

• IDE: Visual Studio Code.

• Interface: console-based user interaction.

1. Requirement Analysis: define user needs and core features.

2. System Design: outline workflow, modules and data structures.

3. Module Development: develop login, food, service, route and feedback modules.

4. Module Integration: combine individual modules into one system.

5. Testing and Debugging: test modules individually and then test the complete user journey.

9. Phase-II Progress and Validation Plan

The supplied source is a Phase-I evaluation document. It identifies the planned modules and team contributions, but it does not provide verified Phase-II completion percentages, test pass/fail results, execution screenshots or confirmed integration status. Therefore, this report lists the validation work required without claiming that tests have passed.

1. Module status: review and record the actual status of login / profile, food selection, dine-in, takeaway / drive-through, delivery, route optimization and feedback.

2. Route validation: run known graph inputs and compare the calculated shortest paths against expected results.

3. Recommendation validation: check whether filtering and comparison use the project factors stated in the plan.

4. Input validation: test valid and invalid user, food-choice, location and service-type inputs.

5. Service-flow validation: test dine-in, takeaway / drive-through and delivery from selection through final output.

6. Integration validation: test the complete user journey after the individual modules are connected.

7. Evidence: add actual screenshots, test inputs, expected outputs, observed outputs and pass/fail results after execution.

10. Roadmap and Expected Outcomes

1. Complete the individual modules.

2. Integrate all modules.

3. Test recommendation and route logic.

4. Test the complete user workflow.

5. Finalize the project and documentation.

• Expected outcome: a C++ console prototype for food selection and route planning.

• Expected outcome: food recommendations based on the project's stated factors.

• Expected outcome: shortest-route calculation.

• Expected outcome: dine-in, takeaway / drive-through and delivery options.

• Expected outcome: login, feedback and order-history features.

11. References

1. B. Stroustrup, *The C++ Programming Language*.

2. Cormen et al., *Introduction to Algorithms*.

3. C++ Standard Library Documentation.

4. E. W. Dijkstra, "A Note on Two Problems Connected with Graphs," 1959.

Note: Module contributions and project details above are carried forward from the supplied Phase-I document. Phase-II progress and test results should be updated only after the team verifies them.